
A A2C RL algorithm to play the game of Stag Hunt with diverse partners

Angelo Terzi

Department of Computer Science
University of Bologna
angelo.terzi2@studio.unibo.it

Abstract

In this Report, we are going to see how the 'Hunt' game from the Stag Hunt Library (<https://github.com/giorgiofranceschelli/Gymnasium-Stag-Hunt>) was approached, with multiple types of players, using RL techniques and DNNs. We implement an Advantage Actor-Critic (A2C) framework with Pytorch, using only one shared actor network across agents and a centralized critic observing the state. To address common issues, such as policy entropy collapse and degenerate stationary equilibria, we introduce entropy regularization - making it effectively a Soft Actor-Critic algorithm - and a distance-to-stag shaping term into the reward signal. We evaluate cooperation under three partner conditions: self-play between two players controlled by the one learned agent, collaboration with a pursuit heuristic agent, and with a random agent.

1 Introduction

MARL, also known as Multiple Agent Reinforcement Learning, has been one of the most important parts of the RL field; this because in the real world, agents rarely act in isolation. However traditional forms of RL often fail to achieve cooperation, as they often lack the global view of the task at hand. In order to overcome this obstacle, I developed an A2C/SAC inspired architecture, following principles of CTDE (although agents have a global view of the system, so it's not to be considered actual Decentralized Execution) to train an agent to cooperate with itself, a random agent, or a pursuit agent (with the latter two being included in the environment themselves).

2 Background

2.1 The Stag Hunt game

"In game theory, the stag hunt (also referred to as the assurance game, trust dilemma or common interest game) describes a situation or game where participants would be better off cooperating to achieve a more ambitious goal (hunting a stag together, which succeeds), but can choose a safer option (hunting a hare on one's own) that protects them from a bad outcome (hunting a stag alone, which fails, while the other hunts a hare)."[1]

2.2 CTDE

Centralized Training, Decentralized Execution is a RL framework that allows for expanded knowledge of a system during training (adding access to other agent(s) observations and actions), so that agents can learn effective collaboration techniques that can then be used at test time, where knowledge of the system may be limited.

2.3 A2C

Advantage Actor-Critic is a specific Actor-Critic technique, a method that learns approximation of both policy and value function. The distinguishing feature of A2C is the so-called "advantage", which describes the difference between the expected cumulative reward we obtain from choosing an action a at state S , and the average at that given state; or, in other words, how better it is to choose action a at state S than to be at state S on average.

$$A(S, a) = Q(S, a) - v(S)$$

This is then used to derive the loss for both the actor and the critic.

3 Implementation

3.1 Network Architecture

3.1.1 Actor

The actor(s) are implemented as a single Neural Network, composed of 1 input layer, 1 hidden layer, and 1 output layer. The three layers are interleaved by ReLUs. The input size is game dependent: in our case, with 2 agents, 1 stag, and 2 plants, we have a 1x10 tensor (each element has an x and a y coordinate). The action space is a discrete one dimensional of size 5 space. Hidden channels are 64 hence the layers become:

- Input layer dimensions: from 1x10 to 1x64
- Hidden layer dimensions: from 1x64 to 1x64
- Output layer dimensions: from 1x64 to 1x5

3.1.2 Critic

The critic is a completely separate instance of a similar Network, which has the same architecture but for the output layer, which by definition has to have a single output.

3.2 Shared Actor

Since the actors see the same observations (with the agents order flipped), it made sense to me to just train one network and use it for both agents (when we had two of them). This has some upsides like having and training only one network, while it had the downside of making the two RL agents perform the same steps as they reached the same square (once we enter test mode at least, since the policy by then becomes deterministic). This is not actually a bad thing, as it may prevent the agents from foraging some hares, but it will also prevent them from being caught alone by the stag.

3.3 Critic

The "centralized critic" is here by design, though it doesn't really provide many benefits over a regular one, since in this case, both agents have full knowledge of the game.

3.4 Entropy

After many attempts at an architecture without entropy, and some debug, I realized that the policy collapsed immediately to simple behaviors (e.g.: two agents standing in a corner and waiting for the stag to get to them).

This was only preventable by introducing entropy, which allowed for a more extensive exploration and finding alternative solutions to the problem, with agents that actually play the game.

3.5 Distance penalty

This is actually a residue of several attempts at dissuading agents from standing together in a corner. It may not be useful right now, though I'm positive it doesn't hurt, and I have no time left to test it's impact on performance, so I left it there.

4 Experiments

The environment chosen was the 'Hunt' one, the one that actually has both the possibility to check for self interest and collaboration. Most parameters are left at default, but the 'stag_follows' one, which was set to "False" in the beginning to try and avoid early collapsing of the policy (the reasoning being that standing in the same corner for both agents provides a very high reward when the stag directly comes to them).

The Network was trained with all kinds of partners available: a learned one, a random one, and a "pursuit" one which only runs towards the stag.

Metrics checked are harvested plants, caught stags, and total reward, to check for both collaboration, selfishness, and effectiveness of the strategy.

4.1 Hyperparameters

Most hyperparameters are left to their default value:

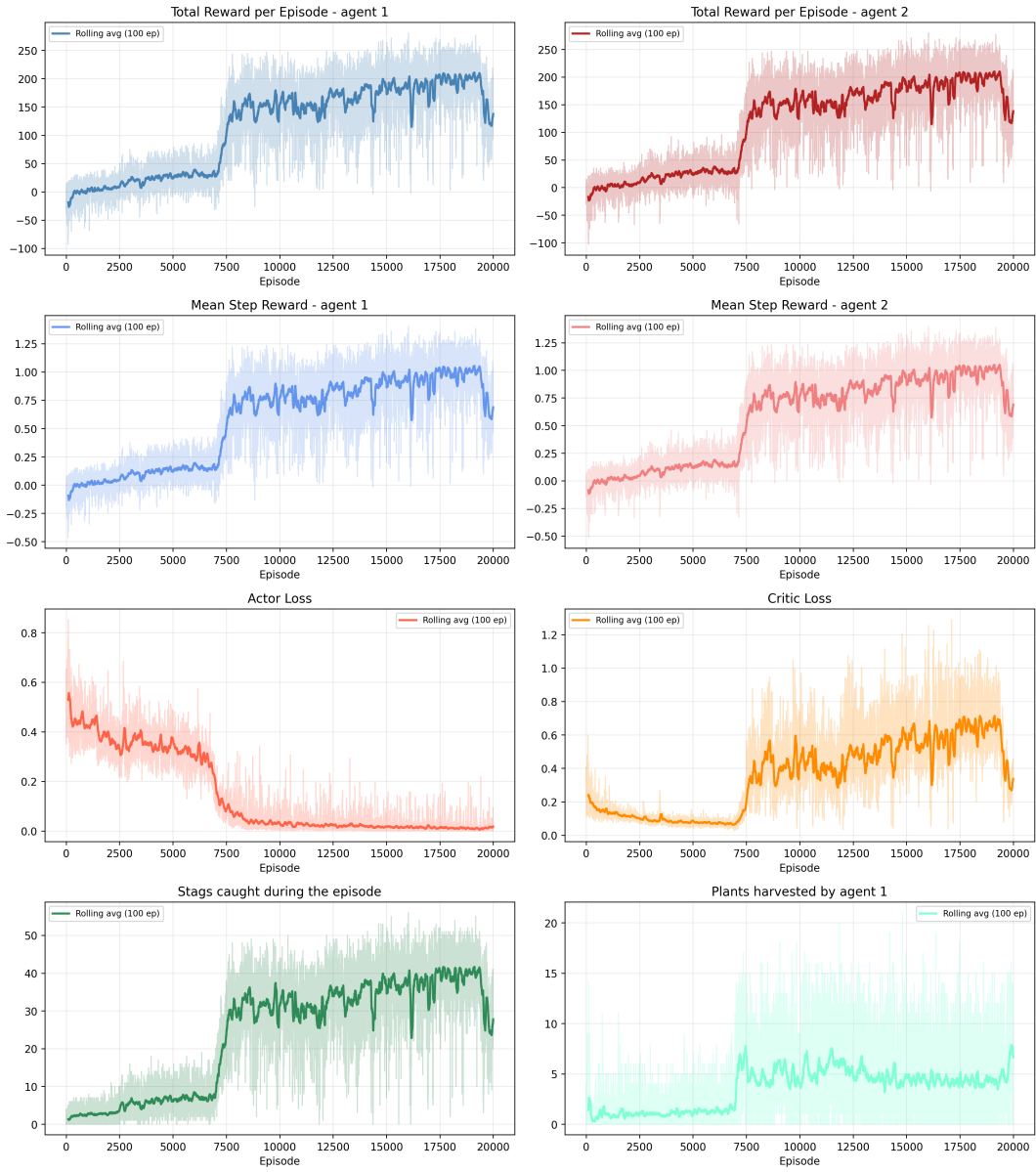
- Gamma: left at 0.99, as the standard recommendation for the TD[0] error used in our A2C agent
- Episodes: started at a standard 10'000, increased to 20'000 to allow for a slower training which got better performance
- Learning rate: 0.001 for Critic, and 0.0005 for Actor
- Distance coefficient: set to 0.1 for pursuit and learned agents, and 0 for random agent (I didn't want to reinforce hunting the stag with a random partner)
- Entropy coefficient: this one was the main difference from the traditional recommendation of '0.001', and was set to $\max(0.001, 0.05 * (0.998 * ep))$, because a non decreasing entropy prevented the policy from becoming close to deterministic eventually.

5 Results

5.1 Learned agents

Below we can see the effectiveness of the technique used through graphs of the training performance. Paying particular attention to the Stags Caught, we can see that collaboration was indeed achieved, even if in a trivial way (both agents performing the same move).

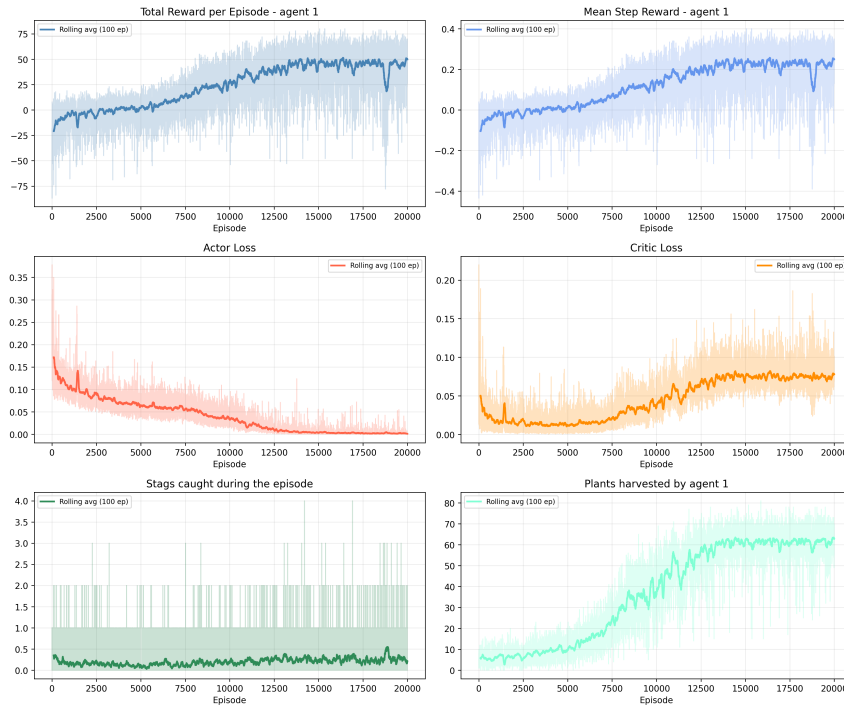
Stag Hunt — A2C Training Curves



5.2 Learned agent and Random agent

Here, with the same graphs, we can see that, even if the cumulative reward increased over time, collaboration with a random agent is effectively impossible, so the agent gave priority to harvesting forage rather than catching stugs.

Stag Hunt — A2C Training Curves



5.3 Learned agent and Pursuit agent

Here we can see how the Learned agent can learn to cooperate with an agent with a predefined strategy.

Stag Hunt — A2C Training Curves

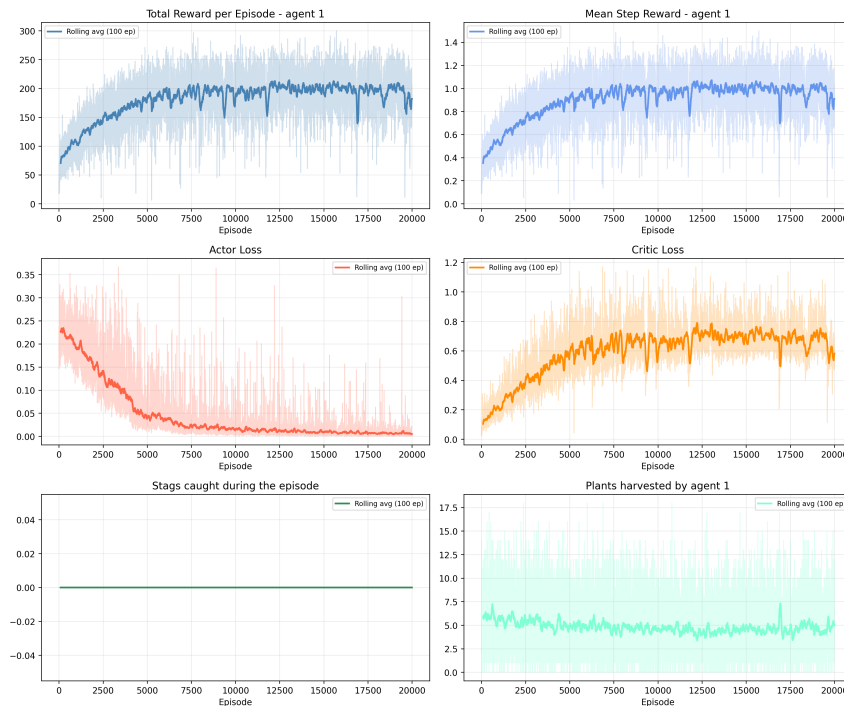


Table 1: Average of test results

Cooperator	Total reward	Stags Caught	Plants harvested
Learned agent	166.81	32.73	4.44
Random agent	50.03	0.21	63.33
Pursuit agent	186.70	37.21	4.85

5.4 Test results

On the table above we can see the average results over a test run of 100 episodes for each companion available. As expected, the worst score was obtained with the Random companion, as no cooperation is possible, hence the score is mostly obtained by harvesting. The second scorer is with the Learned companion, and the best is with the Pursuit agent, as expected.

6 Final remarks

To conclude the report, I can say that the goal was achieved (learn cooperating non-trivial strategies for the Hunt game), though with some caveats! First and most importantly, if I were to do it again, I'd instantiate two actors for the two agents, as to maximize profit of harvesting too. Secondly, had I had the time, I'd have split training in two parts and added environment with following stags! Lastly, as we can see from training graphs, I've chosen a bad amount of episodes (not choosing random seeds until the end has not been the greatest of ideas I've had). Lengthening and shortening training should both fix the loss of performance of the last 1000 episodes or so of training.

7 References

[1] Wikipedia.org - Stag Hunt article

8 Disclaimer

All of the code in the project has been produced by me personally, except for the function of plotting, which initially was produced as AI generated code and later changed multiple times by hand by me personally.